

UNIVERSITY OF ASIA PACIFIC

Department of Computer Science and Engineering

CSE 433: Blockchain & Distributed Security Lab

Comprehensive Codebase Analysis

Peer-to-Peer Network Communication, Framing Protocols, Concurrency Locks, and Distributed File Sharing

Python 3.9+ Standard Library

TCP Socket Architecture

Zero Central Server

Thread-Safe Concurrency

DOCUMENT PURPOSE

Technical Walkthrough & Viva Prep

COMPONENTS ANALYZED

protocol.py, p2p_node.py, main.py, test_p2p.py

TARGET ARCHITECTURE

Full P2P (Server + Client Mesh)

VERIFICATION STATUS

100% Automated Test Suite Passed

1. Executive Overview & Architectural Foundation

1.1 Project Objective (From Assignment Specification)

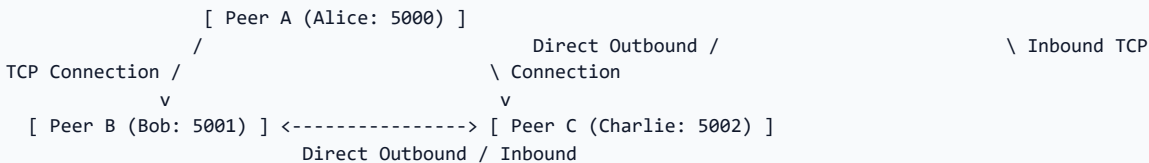
The objective of this assignment is to design and implement a lightweight, fully functional **Peer-to-Peer (P2P) network** in pure Python. The system eliminates reliance on any centralized server, allowing multiple independent nodes (peers) to directly establish TCP connections, execute identity handshakes, exchange real-time text communications, and stream arbitrary binary files (images, audio, video, PDF, ZIP).

1.2 The Fundamental P2P Principle: Dual Role Model

In standard client-server architectures (e.g., HTTP web servers), nodes occupy rigid asymmetric roles: a centralized server remains passive and reachable on a fixed port, while clients initiate outgoing connections. If the server fails, the entire network collapses.

In this P2P network, **every peer executes simultaneously as both a TCP Server and a TCP Client** (Section 8 of the assignment rubric):

- Server Role:** The peer initializes an IPv4 TCP stream socket, binds to `0.0.0.0` on a designated port, enters listening mode, and runs a continuous accept loop to accept incoming connections from other peers.
- Client Role:** The peer can independently dial the IP address and listening port of any other active peer on the local machine (`127.0.0.1`) or across a Wi-Fi/LAN subnet, establishing direct bidirectional byte pipes.



1.3 System Component Structure

Module	Primary Responsibility	Key Architectural Role
<code>protocol.py</code>	Application Wire Protocol	Solves TCP stream boundary ambiguity using 4-byte big-endian framing; defines JSON schemas and chunk sizes.
<code>p2p_node.py</code>	Networking & Concurrency Core	Implements dual server/client sockets, multi-threaded reader loops, per-connection send locks, and two-stage chunked file streaming.
<code>main.py</code>	Graphical User Interface	Tkinter-based GUI; bridges multi-threaded asynchronous networking events to the main UI thread via a thread-safe <code>queue.Queue</code> .
<code>test_p2p.py</code>	Headless Integration Testing	Comprehensive automated test suite covering full-mesh handshakes, SHA-256 integrity validation, path traversal defense, and error handling.
<code>downloads/</code>	Local Storage Repository	Stores received binary files with automatic filename collision resolution and atomic staging via <code>.part</code> extensions.

ZERO DEPENDENCY CONSTRAINT

The entire application is constructed using exclusively the **Python standard library** (`socket`, `threading`, `struct`, `json`, `os`, `uuid`, `queue`, `tkinter`). No external third-party packages are required.

2. Application Wire Protocol: `protocol.py`

2.1 The TCP Byte-Stream Problem & Message Framing

TCP is a byte-stream protocol, not a packet or message protocol. TCP does not preserve application-level boundaries. If a sender transmits two JSON strings sequentially, the receiving socket's `recv()` may return both messages merged together, or return a half-received fragmented chunk.

To establish deterministic message boundaries without relying on delimiter characters (which might appear inside binary payloads), the protocol enforces **Length-Prefixed Framing**:

4-Byte Big-Endian Integer (Len)	UTF-8 Encoded JSON Body
<code>struct.pack("!I", len(payload))</code>	<code>{"type": "text", "message": "Hi!", ...}</code>
<----- 4 Bytes ----->	<----- Len Bytes ----->

2.2 Line-by-Line & Block Analysis of `protocol.py`

Constants and Struct Definition (Lines 27-41)

```
import json
import struct

_HEADER = struct.Struct("!I")
MAX_MESSAGE_SIZE = 1 * 1024 * 1024 # 1 MB
CHUNK_SIZE = 64 * 1024 # 64 KiB
```

- `_HEADER = struct.Struct("!I")`: Compiles a C struct format string. The exclamation point (!) specifies **network byte order (big-endian)**, ensuring cross-platform architecture parity (e.g., x86 vs. ARM). The I format specifies an **unsigned 4-byte integer** (capable of addressing sizes up to 4.29 GB). `_HEADER.size` is strictly 4 bytes.
- `MAX_MESSAGE_SIZE = 1 * 1024 * 1024`: Enforces a strict 1 MB cap on JSON control frames. If a corrupted or hostile sender transmits an exorbitant length (e.g., 2 GB), our node refuses to allocate memory, preventing Denial-of-Service (DoS) and Out-Of-Memory (OOM) fatal crashes.
- `CHUNK_SIZE = 64 * 1024`: Defines the 64 KiB buffer boundary for disk and socket streaming. This guarantees that files of arbitrary size (from 1 KB to 10 GB) are processed in constant memory footprint.

Exact Byte Receiver: `recv_exact(sock, n)` (Lines 43-60)

```
def recv_exact(sock, n):
    data = bytearray()
    while len(data) < n:
        chunk = sock.recv(n - len(data))
        if not chunk: # peer closed the socket
            raise ConnectionError("peer closed the connection")
        data.extend(chunk)
    return bytes(data)
```

- **The Requirement:** Standard `sock.recv(n)` returns between 1 and n bytes depending on OS network buffers. It cannot be assumed to return n bytes.
- `data = bytearray()`: Allocates a mutable byte buffer avoiding repeated memory reallocations during concatenation.
- `while len(data) < n:`: Loops persistently until exactly n bytes have accumulated.
- `chunk = sock.recv(n - len(data))`: Solicits only the missing deficit on each read.

- `if not chunk:`: When a remote peer closes the socket or drops unexpectedly, TCP sends a `FIN` or `RST` packet, causing `recv()` to return an empty byte sequence (`b""`). The function raises `ConnectionError`, triggering clean resource teardown in caller threads.

Frame Serializer: `send_message(sock, message)` (Lines 63-73)

```
def send_message(sock, message):
    payload = json.dumps(message).encode("utf-8")
    frame = _HEADER.pack(len(payload)) + payload
    sock.sendall(frame)
```

Encodes the Python dictionary into a UTF-8 JSON byte string, packs the length into a 4-byte header using `_HEADER.pack()`, concatenates the binary frame, and transmits it via `sock.sendall()`, which blocks until the entire buffer is transferred to the kernel socket buffer.

Frame Deserializer: `recv_message(sock)` (Lines 75-95)

```
def recv_message(sock):
    header = recv_exact(sock, _HEADER.size)
    (length,) = _HEADER.unpack(header)

    if length > MAX_MESSAGE_SIZE:
        raise ValueError("refusing oversize message (%d bytes)" % length)

    body = recv_exact(sock, length)
    try:
        message = json.loads(body.decode("utf-8"))
    except (UnicodeDecodeError, json.JSONDecodeError) as exc:
        raise ValueError("garbage payload: %s" % exc)
    if not isinstance(message, dict):
        raise ValueError("expected a JSON object, got %s" % type(message).__name__)
    return message
```

Reads exactly 4 bytes to determine payload size, unpacks the length, validates it against `MAX_MESSAGE_SIZE`, consumes exactly `length` bytes using `recv_exact`, and deserializes the JSON string back into a Python dictionary.

3. Networking & Concurrency Core: `p2p_node.py`

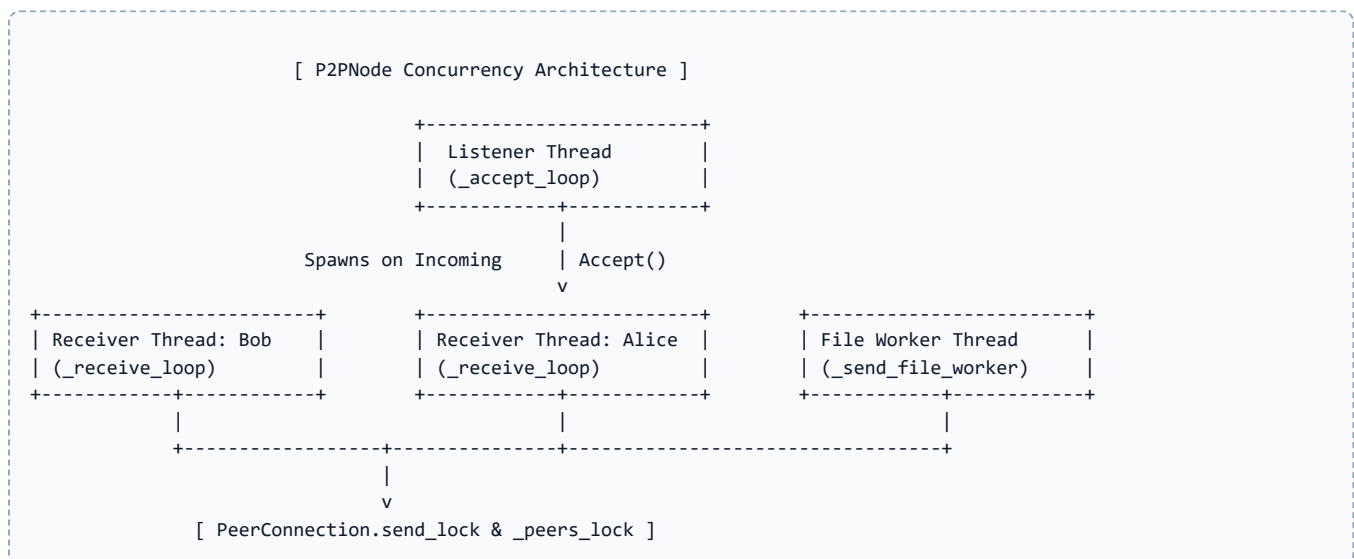
3.1 Concurrency Model & Synchronization Locks

A functional peer must concurrently handle:

1. Listening for new incoming connections from unseen peers.
2. Reading incoming data streams from multiple connected peers simultaneously without blocking.
3. Sending out large file streams in the background without freezing the GUI.

To prevent race conditions, two levels of synchronization locks are implemented:

- `_peers_lock` (Global Node Lock): Protects the `self._peers` dictionary against simultaneous mutations when peers connect, disconnect, or are queried by the UI.
- `send_lock` (Per-Connection Lock): Embedded inside every `PeerConnection` object. Guarantees that chat message frames cannot be interleaved into raw file bytes being streamed across the same socket.



3.2 Step-by-Step Code Analysis of `p2p_node.py`

LAN IP Address Discovery: `get_lan_ip()` (Lines 57-75)

```
def get_lan_ip():
    global _lan_ip_cache
    with _lan_ip_lock:
        if _lan_ip_cache is None:
            probe = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
            try:
                probe.connect(("8.8.8.8", 80))
                _lan_ip_cache = probe.getsockname()[0]
            except OSError:
                _lan_ip_cache = "127.0.0.1"
            finally:
                probe.close()
    return _lan_ip_cache
```

- **Why not** `socket.gethostbyname(socket.gethostname())` ? On Windows, that call often returns `127.0.0.1` or virtual adapter addresses (e.g. VMware, WSL).
- **How it works:** Creates a UDP socket and calls `connect(("8.8.8.8", 80))`. Because UDP is connectionless, *no packet is transmitted over the wire*. The OS simply queries its routing table to select the outgoing network interface and binds the

local adapter. `probe.getsockname()[0]` extracts that physical LAN IP.

The `PeerConnection` Structure (Lines 78-92)

```
class PeerConnection(object):
    __slots__ = ("sock", "peer_id", "peer_name", "ip", "port",
                 "inbound", "send_lock")
    def __init__(self, sock, peer_id, peer_name, ip, port, inbound):
        self.sock = sock
        self.peer_id = peer_id
        self.peer_name = peer_name
        self.ip = ip
        self.port = port
        self.inbound = inbound
        self.send_lock = threading.Lock()
```

Encapsulates socket handles and remote identity. `__slots__` optimizes memory. `send_lock` is dedicated to this specific connection to serialize outgoing writes.

Server Lifecycle: `start()` & `_accept_loop()` (Lines 142-180, 361-369)

```
def start(self, name, port):
    ...
    server_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server_sock.bind(("0.0.0.0", port))
    server_sock.listen(5)
    ...
    self.port = server_sock.getsockname()[1]
    self._running = True
    self._spawn(self._accept_loop)
    return True
```

- `setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)` : Bypasses the operating system's 2-minute `TIME_WAIT` port lock, allowing immediate re-binding if the application is restarted.
- `bind(("0.0.0.0", port))` : Binds to all local network interfaces (both loopback `127.0.0.1` and LAN adapter).
- `listen(5)` : Sets the backlog queue size for pending connection requests to 5.
- `server_sock.getsockname()[1]` : Resolves the actual bound port (crucial when port 0 is passed in automated testing).
- `_spawn(self._accept_loop)` : Launches the connection listening loop in a background daemon thread. Inside the loop, `server_sock.accept()` blocks until an incoming client connects, then spawns `_handle_incoming` on a dedicated thread.

Client Dialing & Application Handshake: `connect_to()` (Lines 208-294)

```
def connect_to(self, ip, port):
    ...
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(self.CONNECT_TIMEOUT)    # 5.0s
    sock.connect((ip, port))

    # Stage 1: Introduce ourselves
    send_message(sock, make_hello(self.peer_id, self.peer_name, self.port))

    # Stage 2: Await remote peer introduction
    ack = recv_message(sock)
    if ack.get("type") != "hello_ack": ...

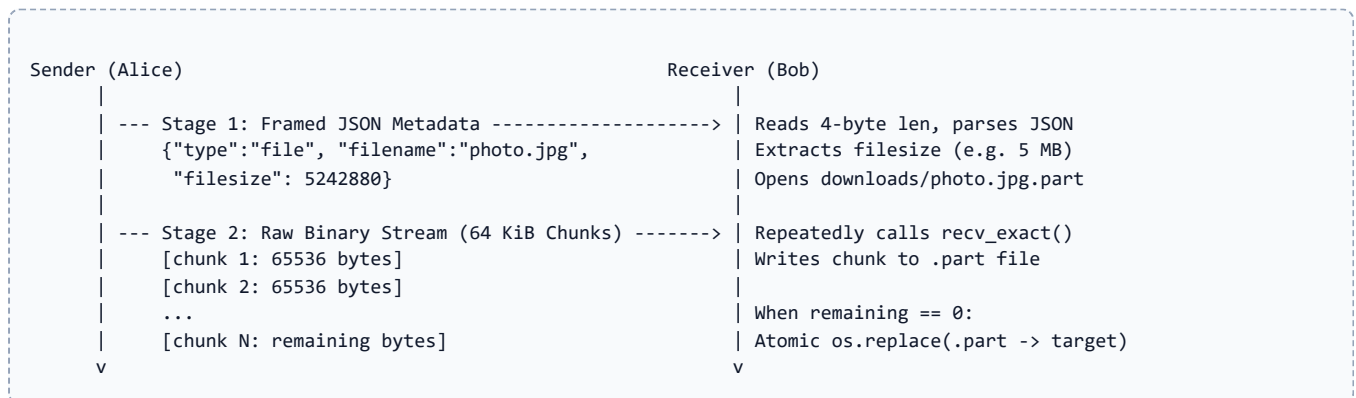
    sock.settimeout(None)                    # Switch to blocking mode
    with self._peers_lock:
        if peer_id in self._peers: ...        # Guard against duplicate connections
        conn = PeerConnection(sock, peer_id, peer_name, ip, peer_port, inbound=False)
        self._peers[peer_id] = conn

    self._on_peers_changed()
    self._spawn(self._receive_loop, conn)
    return True
```

Establishes the TCP connection under a 5-second safety timeout. Immediately transmits a `hello` frame, awaits a `hello_ack` frame, verifies identity, prevents duplicate or self-connections, registers the peer in `self._peers`, and spawns `_receive_loop` to listen for incoming frames.

Two-Stage Chunked File Transfer (Lines 473-550)

File transfer represents the core technical mechanism of the assignment. It operates in two sequential stages across the existing TCP connection:



Key File Security & Robustness Mechanisms in `_receive_file()`:

- Path Traversal Defense:** `safe_name = os.path.basename(raw_name.replace("\", "/"))`. If an attacker sends `"../../evil.bat"`, `os.path.basename` strips parent path references, isolating the file inside `downloads/evil.bat`.
- Atomic File Staging:** Incoming data is written to a temporary `.part` file. Only when all bytes are verified is `os.replace(part_path, target)` executed. If the connection drops midway, the corrupt partial file is quietly deleted.
- Collision Avoidance:** `_unique_path()` inspects the download directory. If `photo.jpg` exists, it writes to `photo (1).jpg`, preventing accidental data overwrites.

4. Graphical User Interface: `main.py`

4.1 The Tkinter Thread-Safety Problem & The Event Queue

CRITICAL GUI CONSTRAINT

Tkinter is a single-threaded wrapper around Tcl/Tk. **Widgets may ONLY be accessed or modified from the main GUI thread.** If a background network thread directly calls `listbox.insert()` or `text.configure()`, Python will crash with segmentation faults, throw Tcl interpreter errors, or experience deadlocks.

To resolve this, `main.py` implements a **Producer-Consumer Event Queue pattern**:

```
[ Network Worker Threads ]      [ Queue Buffer ]      [ Tkinter Main Thread ]
Receiver Thread (Alice)  ---Receiver Thread (Bob)  ----> [ queue.Queue() ] <--- Polls every 100ms via root.after()
File Worker Thread      ---/                      Updates Widgets & Event Logs
```

4.2 Step-by-Step Code Analysis of `main.py`

Initialization & Polling Loop (Lines 31-49)

```
class ChatApp(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("P2P Chat & File Sharing - CSE 433")
        self.geometry("%dx%d" % (WINDOW_W, WINDOW_H))
        self.node = None
        self.peer_ids = {}
        self.event_queue = queue.Queue()

        self._build_widgets()
        self.after(100, self._drain_events)
        self.protocol("WM_DELETE_WINDOW", self._on_close)
```

Configures window dimensions and initializes state. `self.after(100, self._drain_events)` registers a non-blocking recurring callback in the Tkinter event loop to drain the queue every 100 milliseconds.

Event Draining Routine (Lines 268-285)

```
def _drain_events(self):
    self._drain_after_id = None
    try:
        while True:
            kind, payload = self.event_queue.get_nowait()
            if kind == "log":
                self._append_log(payload)
            elif kind == "peers":
                self._refresh_peers()
            elif kind == "file_sent":
                peer_name, filename = payload
                self._append_log("You -> %s: File sent: %s" % (peer_name, filename))
                self.status_var.set("File sent to %s" % peer_name)
    except queue.Empty:
        pass
    self._drain_after_id = self.after(100, self._drain_events)
```

Consumes all pending events non-blockingly using `get_nowait()`. Updates the text log, refreshes peer lists, updates status indicators, and reschedules itself for the next tick.

Callback Hooking in `_start_peer()` (Lines 183-186)

```
node = P2PNode(downloads_dir="downloads",
               on_log=lambda line: self.event_queue.put(("log", line)),
               on_peers_changed=lambda: self.event_queue.put(("peers", None)),
               on_file_sent=lambda p, f: self.event_queue.put(("file_sent", (p, f))))
```

Decouples the networking node completely from Tkinter. Background threads emit callbacks that simply execute thread-safe `event_queue.put()` calls.

5. Automated Integration Test Suite: `test_p2p.py`

5.1 Test Methodology

The test suite in `test_p2p.py` validates all 13 testing criteria from Section 13 of the assignment without launching graphical interfaces. It executes pure network integration tests by running multiple `P2PNode` instances in memory on loopback (`127.0.0.1`) using dynamic ephemeral OS ports (port 0).

5.2 Test Scenarios Overview

Test Method	Tested Scenario	Verification Mechanism
<code>test_handshake_and_peer_lists</code>	Full mesh connection between 3 nodes (Alice, Bob, Charlie).	Validates bidirectional peer ID discovery and port tracking.
<code>test_text_messages</code>	Bidirectional text message propagation.	Asserts payload delivery and message order across nodes.
<code>test_file_transfers</code>	Transfers of small text, ~5 MB binary blob, 0-byte file, and Unicode filename.	Calculates SHA-256 cryptographic hashes of source and destination files to verify bitwise parity.
<code>test_text_right_after_big_file</code>	Framing & <code>send_lock</code> stress test: Text → 5 MB File → Text.	Validates that messages do not interleave and arrive in exact sequence.
<code>test_abrupt_disconnect_survival</code>	Node crashes while peers are connected.	Surviving nodes detect socket drop, remove peer, and continue communicating.
<code>test_connection_errors</code>	Invalid IP, closed ports, self-connection, duplicate connection.	Clean error handling without uncaught thread exceptions.
<code>test_malicious_filename</code>	Path traversal attack (<code>../../evil.txt</code>).	Asserts file is sanitized and confined strictly to <code>downloads/</code> .
<code>test_garbage_data_is_rejected</code>	Remote peer sends 4 GB length prefix.	Oversize check triggers, drops rogue connection, node survives.

6. Complete Viva Examination Guide (20 Questions)

Q1. What is the difference between a client and a server?

A **server** passively listens for connections by calling `socket()`, `bind()`, `listen()`, and `accept()`. A **client** actively initiates a connection by calling `socket()` and `connect()`. Once established, the connection is a symmetrical bidirectional byte pipe.

Code: Server in `P2PNode.start() / _accept_loop()`; Client in `P2PNode.connect_to()`

Q2. In this project, why can one peer act as both a client and a server?

Because there is no central server. Every node must be capable of accepting incoming connections from other peers (server functionality) while also being able to initiate connections to other peers (client functionality).

Code: Every peer runs `P2PNode` with both `_server_sock` listener and `connect_to()` client

Q3. What is the purpose of an IP address?

An IP address uniquely identifies a network interface card (host computer) on an IP network, allowing routers and switches to direct packets to the correct machine.

Code: Passed into `sock.connect((ip, port))` and reported via `get_lan_ip()`

Q4. What is the purpose of a port number?

A port number identifies a specific application or process running on a host computer. It multiplexes network traffic so multiple network applications can operate simultaneously on the same IP.

Code: `server_sock.bind(("0.0.0.0", port))` in `p2p_node.py`

Q5. Why do different peers need different ports when testing on the same computer?

An operating system does not permit two processes to bind to the same IP address and port number simultaneously. When running multiple peers on `127.0.0.1`, each peer must bind to a distinct port (e.g., 5000, 5001, 5002).

Code: Configured via "Port:" inputs in GUI or port 0 in automated tests

Q6. What happens when `connect()` is called?

The operating system initiates the **TCP 3-way handshake** (`SYN` → `SYN-ACK` → `ACK`) with the destination address. Once acknowledged, `connect()` returns, establishing a reliable, full-duplex byte stream.

Code: `sock.connect((ip, port))` in `P2PNode.connect_to()`

Q7. What happens when `accept()` is called?

`accept()` blocks until an incoming client completes the TCP handshake. It extracts the connection from the listen queue and returns a **new socket** dedicated to that client, along with the client's IP and port. The original listening socket remains open.

Code: `conn, addr = self._server_sock.accept()` in `_accept_loop()`

Q8. Why is TCP used instead of UDP?

TCP provides guaranteed, in-order, error-checked delivery of bytes with automated packet retransmission. Binary files and JSON control messages cannot tolerate packet loss or out-of-order delivery, which UDP does not prevent.

Code: `socket.SOCK_STREAM` in `protocol.py` and `p2p_node.py`

Q9. Why are threads used?

Socket operations (`accept()`, `recv()`) are blocking. Without multi-threading, waiting for data from Alice would freeze the entire application, preventing the node from receiving messages from Bob or responding to the GUI.

Code: `threading.Thread(target=..., daemon=True)` via `_spawn()`

Q10. What is the purpose of the HELLO message?

It acts as the **application-level handshake**. While TCP establishes the low-level connection, the application needs to learn the remote peer's identity (peer ID, display name, and remote listening port) before normal communication begins.

Code: `make_hello()` and `make_hello_ack()` in `protocol.py`

Q11. What is message framing?

TCP is a continuous byte stream without message boundaries. Framing is the technique of prefixing or delimiting messages so the receiver can delineate where one message ends and the next begins. Our protocol prepends a 4-byte length header.

Code: `_HEADER.pack(len(payload))` in `send_message()`

Q12. Why can't we assume that one `recv()` call gives us one complete message?

Due to network packet fragmentation, MTU limits, and OS buffering, a single message may arrive in multiple small fragments, or multiple messages may arrive concatenated in a single `recv()` buffer.

Code: Resolved by `recv_exact(sock, n)` in `protocol.py`

Q13. Why is file metadata sent before the actual file?

The receiver must know the filename and exact file size in bytes before the binary stream begins. The file size tells the receiver exactly how many raw bytes to consume before returning to JSON message mode.

Code: `send_message(peer.sock, header)` in `_send_file_worker()`

Q14. Why are files transferred in chunks?

Loading a large file (e.g., 2 GB) entirely into memory via `read()` would exhaust RAM and cause performance degradation. Streaming in 64 KiB chunks keeps RAM usage fixed at a few kilobytes regardless of file size.

Code: `CHUNK_SIZE = 64 * 1024` in `protocol.py`

Q15. How does the receiver know when the file transfer has finished?

The receiver extracts `filesize` from the metadata and maintains a `remaining` byte counter. It loops `recv_exact()` until exactly `filesize` bytes have been read. It does not rely on socket closure.

Code: `while remaining > 0: in _receive_file()` in `p2p_node.py`

Q16. What happens if the other peer disconnects?

The active `recv()` call returns an empty byte string (`b""`), triggering a `ConnectionError`. The node catches this, closes the socket, removes the peer from `_peers`, updates the UI, and deletes any partial `.part` file.

Code: `_drop_peer()` and `_quiet_remove(part_path)` in `p2p_node.py`

Q17. Where is the received file stored?

In the local `downloads/` directory, safely sanitized against directory traversal attacks and renamed if a file with the same name already exists.

Code: `self.downloads_dir` in `p2p_node.py`

Q18. Is there a central server in this system?

No. The network is completely decentralized. Peers establish direct peer-to-peer TCP socket connections without routing traffic through any intermediary server.

Q19. What would happen if the central server in a client-server system went offline?

The entire system would suffer complete outage (Single Point of Failure). In contrast, in a P2P network, if any individual peer goes offline, all remaining peers continue communicating without disruption.

Q20. What additional challenges must be solved for large-scale Internet P2P?

1. **NAT Traversal:** Consumer routers block inbound connections; requires STUN, TURN, or UPnP.
2. **Peer Discovery:** Requires Distributed Hash Tables (DHT like Kademlia) or bootstrap nodes.
3. **Security:** Requires End-to-End Encryption (TLS) and digital signatures for authentication.
4. **Churn Handling:** Managing high rates of peers joining and leaving dynamically.

7. Step-by-Step Live Demonstration Protocol

7.1 Demonstration on a Single Computer

1. **Launch Instance 1:** Open Terminal 1 and run `python main.py`.
 - Set Name: `Alice`, Port: `5000`. Click **Start Peer**.
 - Notice header displays: `Alice | ID: [8-hex-chars] | Port: 5000`.
2. **Launch Instance 2:** Open Terminal 2 and run `python main.py`.
 - Set Name: `Bob`, Port: `5001`. Click **Start Peer**.
3. **Connect Bob to Alice:**
 - In Bob's window, set IP: `127.0.0.1`, Port: `5000`. Click **Connect**.
 - Both windows instantly refresh their **Connected Peers** list.
4. **Demonstrate Text Messaging:**
 - In Bob's window, select `Alice` in the list, type `"Hello Alice!"`, and click **Send**.
 - Alice's event log displays: `Bob -> You: Hello Alice!`.
5. **Demonstrate File Transfer:**
 - In Alice's window, select `Bob`, click **Choose File & Send**, and select an image or video file.
 - Bob's log displays: `Alice -> You: File received: [filename]`.
 - Open the `downloads/` folder in File Explorer to show the received file intact.
6. **Demonstrate Multi-Peer Mesh (Third Node):**
 - Open Terminal 3: `python main.py`. Name: `Charlie`, Port: `5002`. Click **Start Peer**.
 - Connect Charlie to Alice (`127.0.0.1:5000`) and Charlie to Bob (`127.0.0.1:5001`).
 - Show that all peers maintain active connections with each other simultaneously.

7.2 Demonstration Across Two Computers on the Same LAN / Wi-Fi

1. Connect both laptops to the same Wi-Fi network.
2. On Computer A, launch `python main.py`. Note the LAN IP in the bottom right corner (e.g. `192.168.1.15`). Start peer on port `5000`.
3. On Computer B, launch `python main.py`. Set IP to Computer A's LAN IP (`192.168.1.15`) and port to `5000`. Click **Connect**.
4. Exchange text and binary files across the physical network.

WINDOWS FIREWALL TIP

If connecting across two PCs fails with "Connection timed out" or "Connection refused", ensure Windows Defender Firewall allows Python through on **Private Networks** on the listening PC.